



Protocol Audit Report

Prepared by: Chijulybuilds

Prepared by: Chijulybuilds

Lead Auditors:

- Chijulybuilds

Table of Contents

- [Protocol Summary](#)
- [Disclaimer](#)
- [Risk Classification](#)
- [Audit Details](#)
 - [Scope](#)
 - [Roles](#)
- [Executive Summary](#)
 - [Issues found](#)
- [Findings](#)
- [High](#)
- [Medium](#)
- [Low](#)
- [Informational](#)
- [Gas](#)

Protocol Summary

PasswordStore Protocol serves to give exclusively ownership and security to contract owners. The protocol allows an owner to set and retrieve a password, with the intended business logic that only the contract owner should be able to modify the stored password.

Disclaimer

The Chijulybuilds team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the

underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the [CodeHawks](#) severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond to the following commit hash:

```
Commit hash: 2e8f81e263b3a9d18fab4fb5c46805ffc10a9990
```

Scope

```
./src/  
## PasswordStore.sol
```

Roles

- **Owner:** The user who can set and retrieve the password.
- **Attackers:** No one else should have access to modify the password.

Executive Summary

I spent over 5 hours in total time metrics checking for vulnerabilities, testing severity, and writing my findings.

Issues found

Severity	Issues Found
High	2
Medium	0
Low	0
Info	0
Total	2

Findings

High

[S-1] Variables stored in private storage on-chain are visible to anyone, making the password insecure

Description:

The `PasswordStore::s_password` private state variable is still visible to anyone on-chain. The `private` visibility keyword in Solidity only prevents other smart contracts from reading or modifying the variable internally. It does not prevent external users or off-chain observers from reading the data. Since every variable occupies a predictable storage slot in the EVM, an attacker can directly query the storage slot using an RPC provider.

An attacker can easily extract the password using the following command:

```
cast storage <CONTRACT_ADDRESS> <STORAGE_SLOT_NUM> --rpc-url <RPC_URL>
```

Impact:

High. The core business logic and primary objective of the protocol (keeping a password secure and private) are entirely broken.

Proof of Concept:

- ## 1. Start a local development chain (Anvil):

```
make anvil
```

- ## 2. Deploy the PasswordStore contract:

```
make deploy
```

3. Query storage slot 1 of the deployed contract directly from the EVM layer:

```
cast storage 0x5fbdb2315678afecb367f032d93f642f64180aa3 1 --rpc-url $ANVIL URL
```

4. Parse the resulting `bytes32` hex data into a human-readable string:

```
cast parse-bytes32-string 0x6d7950617373776f726400000000000000000000000000000000000000000000
```

Recommended Mitigation:

Smart contracts should never store raw sensitive values or plain-text secrets in on-chain storage. If the protocol must verify a password, consider moving the logic to an off-chain architecture, or store a cryptographic hash instead of plain text.

For example, modify the contract to store a `bytes32` hash, and have the `setPassword` function hash the input using `keccak256`:

```
event SetNetPassword();
bytes32 private s_password;

function setPassword(string memory newPassword) external {
    s_password = keccak256(abi.encodePacked(newPassword));
    emit SetNetPassword();
}
```

[S-2] `PasswordStore::setPassword()` lacks access control, allowing anyone to modify the password

Description:

The protocol's intended business logic is to restrict password updates exclusively to the contract owner. However,

`PasswordStore::setPassword()` lacks any access control checks or modifiers. Any external caller or attacker can invoke this function and overwrite the stored password.

```
function setPassword(string memory newPassword) external {
    // @audit Missing access control check
    s_password = newPassword;
    emit SetNetPassword();
}
```

Impact:

High. Any unauthorized user can alter the protocol's state and modify the password at any time.

Proof of Concept:

Via Cast (Command Line)

1. Start a local development chain:

```
make anvil
```

2. Deploy the contract using the primary Anvil account:

```
forge script script/DeployPasswordStore.s.sol:DeployPasswordStore --rpc-url $ANVIL_U
```

3. Call `setPassword` using a different, unauthorized private key:

```
cast send $CONTRACT_ADDRESS "setPassword(string)" "STRANGE_PASSWORD" --private-key $
```

Via Foundry Test Suite

```
function test_anyone_can_set_password(address randomAddress) public {
    vm.assume(randomAddress != owner);
    vm.prank(randomAddress);
    string memory expectedPassword = "myNewPassword";
    passwordStore.setPassword(expectedPassword);
}
```

```
vm.prank(owner);
string memory actualPassword = passwordStore.getPassword();
assertEq(actualPassword, expectedPassword);
}
```

Recommended Mitigation:

Restructure the `setPassword` function to restrict access strictly to the owner. This can be achieved using either of the following patterns:

Option A: Implement an `onlyOwner` modifier

```
error Password_NotOwner();

modifier onlyOwner() {
    if (msg.sender != s_owner) {
        revert Password_NotOwner();
    }
    _;
}

function setPassword(string memory newPassword) external onlyOwner {
    s_password = newPassword;
    emit SetNetPassword();
}
```

Option B: Add an explicit conditional check

```
error Password_NotOwner();

function setPassword(string memory newPassword) external {
    if (msg.sender != s_owner) {
        revert Password_NotOwner();
    }
    s_password = newPassword;
    emit SetNetPassword();
}
```

Medium

No medium severity findings identified.

Low

No low severity findings identified.

Informational

No informational findings identified.

Gas

No gas-related findings identified.